DSC 40B

Theoretical Foundations II

Lecture 12 | Part 1

Time Complexity of BFS

Full BFS

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            bfs(graph, node, status)


def bfs(graph, source, status=None):
    """Start a BFS at `source`."""
    if status is None:
        status = {node: 'undiscovered' for node in graph.nodes}

    status[source] = 'pending'
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                # append to right
                pending.append(v)
        status[u] = 'visited'
```

Full BFS (“Inlined”)

```
def full_bfs(graph):
    status = {node: 'undiscovered' for node in graph.nodes}
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            # BFS starting at `node`
            status[node] = 'pending'
            pending = deque([node])

            # while there are still pending nodes
            while pending:
                u = pending.popleft()
                for v in graph.neighbors(u):
                    # explore edge (u,v)
                    if status[v] == 'undiscovered':
                        status[v] = 'pending'
                        # append to right
                        pending.append(v)
                status[u] = 'visited'
```



Full BFS ("Simplified")

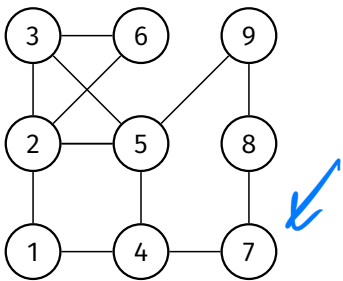
```
def full_bfs(graph):
    ...
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            ...
            while pending:
                u = pending.popleft()
                for v in graph.neighbors(u):
                    # explore edge (u,v)
                    ...
                ...
```

Executed |V| times overall.
Constant time per exec.
So: $\Theta(V)$ time

Exercise

Consider an arbitrary node u. How many times is u added to the queue in a full BFS of the graph below?

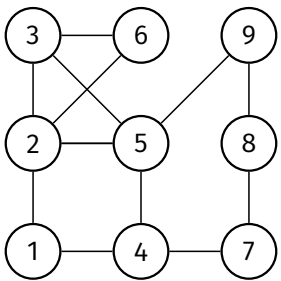
A) once B) twice C) degree(u)



Exercise

Consider an arbitrary node u. How many times is u *popped* from the queue in a full BFS of the graph below?

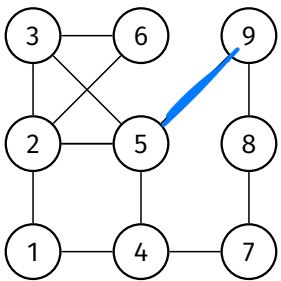
A) once B) twice C) degree(u)



Exercise

Consider an arbitrary edge (u, v). How many times is (u, v) "explored" in a full BFS of the graph below?

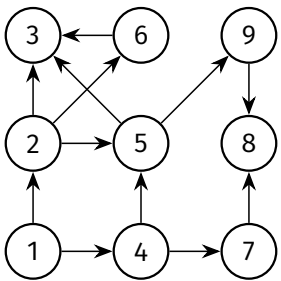
A) once B) twice C) can't say



Exercise

Consider an arbitrary edge (u, v). How many times is edge (u, v) "explored" in a BFS of the *directed* graph below?

A) once B) twice C) can't say



Key Properties of `full_bfs`

- Each node added to queue and popped from queue **exactly once**.
- Each edge is explored **exactly**:
  - **once** if graph is **directed**.
  - **twice** if graph is **undirected**.
- These properties help in doing an **aggregate analysis**.

Full BFS ("Simplified")

```
def full_bfs(graph):
    ...
    for node in graph.nodes:
        if status[node] == 'undiscovered':
            ...
            while pending:
                u = pending.popleft()
                for v in graph.neighbors(u):
                    # explore edge (u,v)
                    ...
                ...
```

$\Theta(V)$ overall

$\Theta(1)$ time to exec once.

Executed:

$|E|$ times overall, if G directed

$2|E|$ times overall if G undirected

Time taken: $\Theta(E)$

$\Theta(V+E)$

Time Complexity

- Full BFS takes $\Theta(V + E)$

Time Complexity

- Full BFS takes $\Theta(V + E)$
- Why not just $\Theta(E)$?
- $\Theta(V + E)$ works *for all graphs*.
  - If we know more about the number of edges, we might be able to simplify.

Live Q&A

directed: $|E| = |V|^2$
undirected: $|E| = \frac{|V|(|V|-1)}{2}$

$\Theta(V^2)$

Exercise

A **complete graph** is a graph in which every pair of nodes is connected by an edge.

What is the time complexity of a full BFS on a complete graph with $|V|$ nodes?

A) $\Theta(V)$ B) $\Theta(V \log V)$ C) $\Theta(V^2)$ D) $\Theta(V^3)$

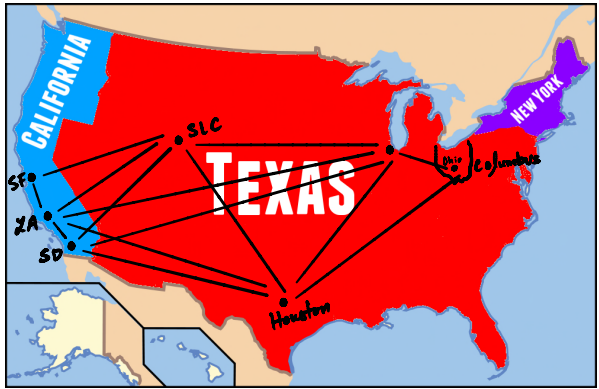
$\Theta(V+E) \rightarrow \Theta(V+V^2) \rightarrow \Theta(V^2)$

DSC 40B

Theoretical Foundations II

Lecture 12 | Part 2

Shortest Paths


CALIFORNIA
NEW YORK
TEXAS
SLC
SF
LA
SD
Houston
Ohio
Columbus

Recall

- The **length** of a path is

  (# of nodes) – 1

Definitions

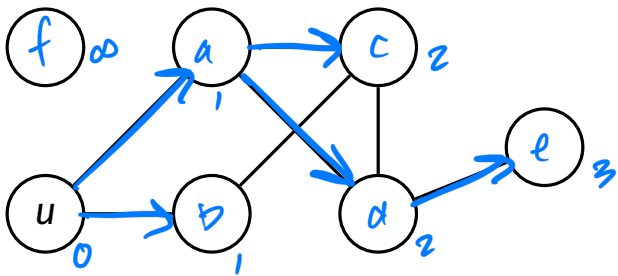
- A **shortest path** between u and v is a path between u and v with smallest possible length.
  - There may be several, or none at all.

- The **shortest path distance** is the length of a shortest path.
  - Convention: ∞ if no path exists.
  - "the distance between u and v" means spd.

Today: Shortest Paths

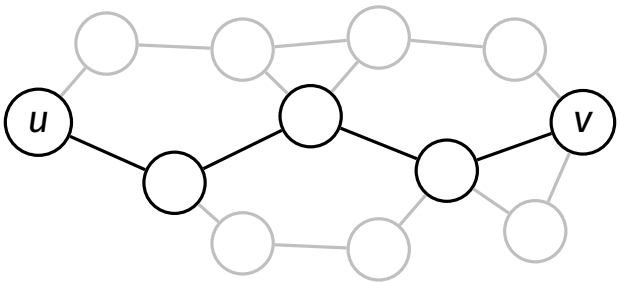
- **Given**: directed/undirected graph G, source u
- **Goal**: find shortest path from u to every other node

Example



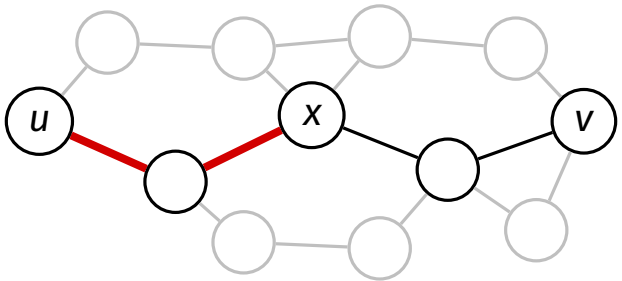
Key Property of Shortest Paths

- Shortest paths are made up of shortest paths.

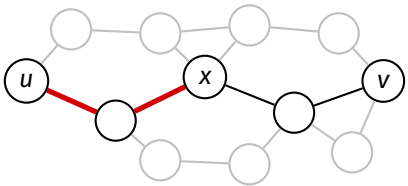


Key Property of Shortest Paths

- If a shortest path from *u* to *v* goes through *x*, then the part of it from *u* to *x* is a **shortest path** from *u* to *x*.



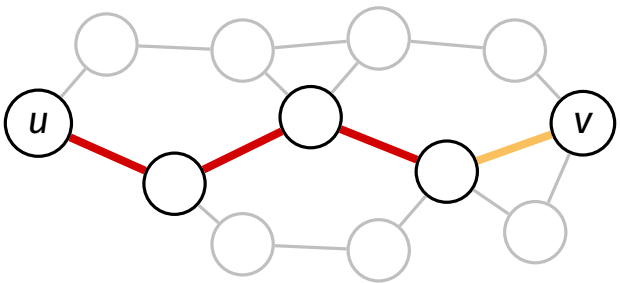
Proof (by contradiction)



- Suppose not.
- That is, suppose there is an even shorter path from *u* to *x*.
- Then replace that part of the path from *u* to *v* with this shorter path.
- This gives a path from *u* to *v* that is even shorter, contradicting your assumption that the original path was *shortest*.

Key Property, Restated

- A shortest path from *u* to *v* of length *k* is composed of:
  - A **shortest path** of length *k* – 1 to one of *v*'s neighbors.
  - Plus one edge.

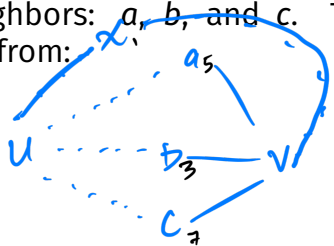


Live Q&A

Exercise

Node v has three neighbors: a, b, and c. The shortest path distance from:

- u to a is 5.
- u to b is 3.
- u to c is 7.



What is the shortest path distance from u to v?

A) 3 B) 4 C) 5 D) 8 E) 2

Algorithm Idea

- Find all nodes distance 1 from source.
- Use these to find all nodes distance 2 from source.
- Use these to find all nodes distance 3 from source.
- ...

It turns out...

...this is exactly what BFS does.

DSC 40B

Theoretical Foundations II

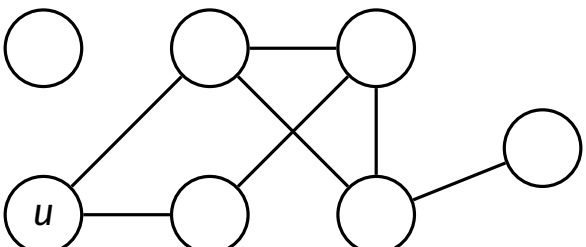
Lecture 12 | Part 3

BFS for Shortest Paths

Key Property of BFS

- For any $k \geq 1$ you choose:
- All nodes distance $k - 1$ from source are added to the queue before any node of distance k.
- Therefore, nodes are “processed” (popped from queue) in order of distance from source.

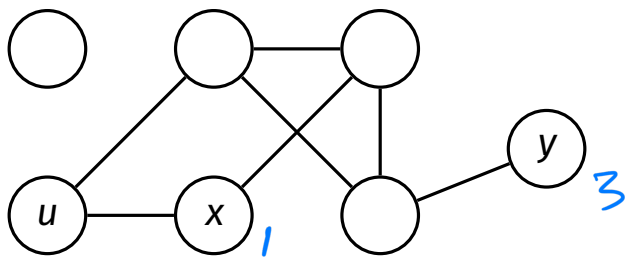
Example



Exercise

Consider a BFS of the graph below, starting at u.

True or False: it is possible that at some point during BFS, x and y are in the queue at the same time.



Discovering Shortest Paths

- We “discover” shortest paths when we pop a node from queue and look at its neighbors.
- But the neighbor’s status matters!

Consider This

- We pop a node *s*.
- It has a neighbor *v* whose status is **undiscovered**.
- We've discovered a **shortest path** to *v* through *s*!

Consider This

- We pop a node *s*.
- It has a neighbor *v* whose status is **pending** or **visited**.
- We **already** have a shortest path to *v*.

Modifying BFS

- Use BFS “framework”.
- Return dictionary of **search predecessors**.
  - If v is discovered while visiting u, we say that u is the **BFS predecessor** of v.
  - This encodes the shortest paths.
- Also return dictionary of shortest path distances.

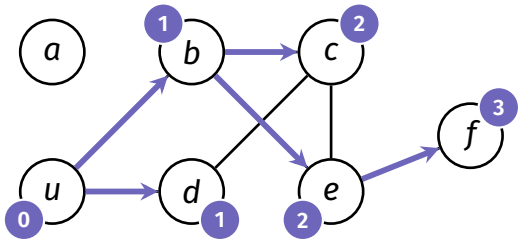
```python
def bfs_shortest_paths(graph, source):
    """Start a BFS at `source`."""
    status = {node: 'undiscovered' for node in graph.nodes}
    distance = {node: float('inf') for node in graph.nodes}
    predecessor = {node: None for node in graph.nodes}

    status[source] = 'pending'
    distance[source] = 0
    pending = deque([source])

    # while there are still pending nodes
    while pending:
        u = pending.popleft()
        for v in graph.neighbors(u):
            # explore edge (u,v)
            if status[v] == 'undiscovered':
                status[v] = 'pending'
                distance[v] = distance[u] + 1
                predecessor[v] = u
                # append to right
                pending.append(v)
        status[u] = 'visited'

    return predecessor, distance
```

Example




```
predecessor = {
    'u': None,
    'a': None,
    'b': 'u',
    'c': 'b',
    'd': 'u',
    'e': 'b',
    'f': 'e'
}

distance = {
    'u': 0, 'a': inf,
    'b': 1, 'c': 2,
    'd': 1, 'e': 2,
    'f': 3
}
```

DSC 40B

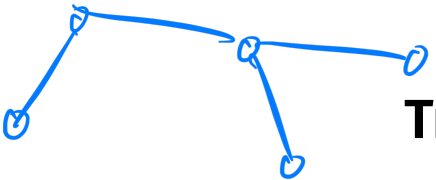
Theoretical Foundations II

Lecture 12 | Part 4

BFS Trees

Result of BFS

- Each node reachable from source has a single BFS predecessor.
  - Except for the source itself.
- The result is a **tree** (or forest).



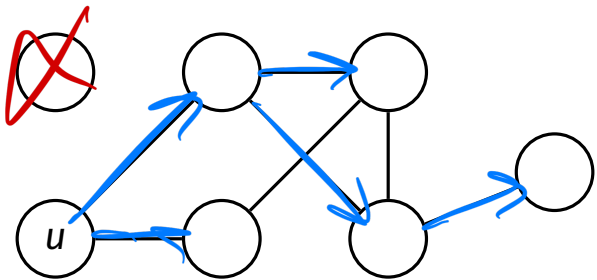
Trees

- A (free) **tree** is an undirected graph $T = (V, E)$ such that T is connected and $|E| = |V| - 1$.
- A **forest** is a graph in which each connected component is a tree.

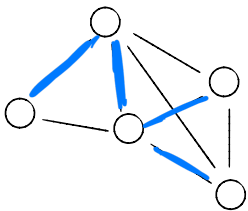
BFS Trees (Forests)

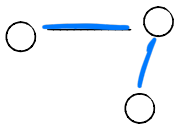
- If the input is connected, BFS produces a **tree**.
- If the input is not connected, BFS produces a **forest**.

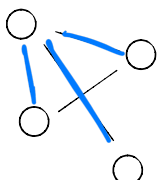
Example



Example







BFS Trees

- BFS trees and forests encode shortest path distances.